LAB MANUAL
PART B

Roll No. C-60	**Name: Avdhut Madhavi**
Class. SE-C	**Batch: C2**
Date of Experiment:	**Date of Submission:**
Grade:	

B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.

- Modify the program to also print the list of jobs that were rejected due to deadline conflicts, and record the output for a given set of jobs.

```
#include <stdio.h>
#include <conio.h>

struct Job {
   char id;
   int deadline;
   int profit;
};

void jobSequencing(struct Job jobs[], int n) {
   struct Job temp, rejected[20];
   int slot[20], i, j, maxDeadline;
   int totalProfit, count, rejectedCount;
   int scheduled, start;
   char result[20];

   /* Sort jobs by profit in descending order */
   for (i = 0; i < n - 1; i++) {
      for (j = 0; j < n - i - 1; j++) {
         if (jobs[j].profit < jobs[j + 1].profit) {
            temp = jobs[j];
            jobs[j] = jobs[j + 1];
            jobs[j + 1] = temp;
         }
      }
   }

   maxDeadline = 0;

   for (i = 0; i < n; i++) {
      if (jobs[i].deadline > maxDeadline)
         maxDeadline = jobs[i].deadline;
   }
```

```
if (maxDeadline > n)
   maxDeadline = n;

for (i = 1; i <= maxDeadline; i++)
   slot[i] = 0;

totalProfit = 0;
count = 0;
rejectedCount = 0;

/* Schedule jobs */
for (i = 0; i < n; i++) {
   scheduled = 0;
   start = jobs[i].deadline;

   if (start > maxDeadline)
      start = maxDeadline;

   for (j = start; j >= 1; j--) {
      if (slot[j] == 0) {
         slot[j] = 1;
         result[j] = jobs[i].id;
         totalProfit += jobs[i].profit;
         count++;
         scheduled = 1;
         break;
      }
   }

   if (scheduled == 0) {
      rejected[rejectedCount] = jobs[i];
      rejectedCount++;
   }
}

printf("\nScheduled jobs: ");

for (i = 1; i <= maxDeadline; i++) {
   if (slot[i] != 0)
      printf("%c ", result[i]);
}

printf("\nTotal jobs done = %d\n", count);
printf("Total profit = %d\n", totalProfit);

printf("Rejected jobs due to deadline conflicts: ");

if (rejectedCount == 0) {
   printf("None");
} else {
   for (i = 0; i < rejectedCount; i++)
      printf("%c ", rejected[i].id);
}
```

```
    printf("\n");
}

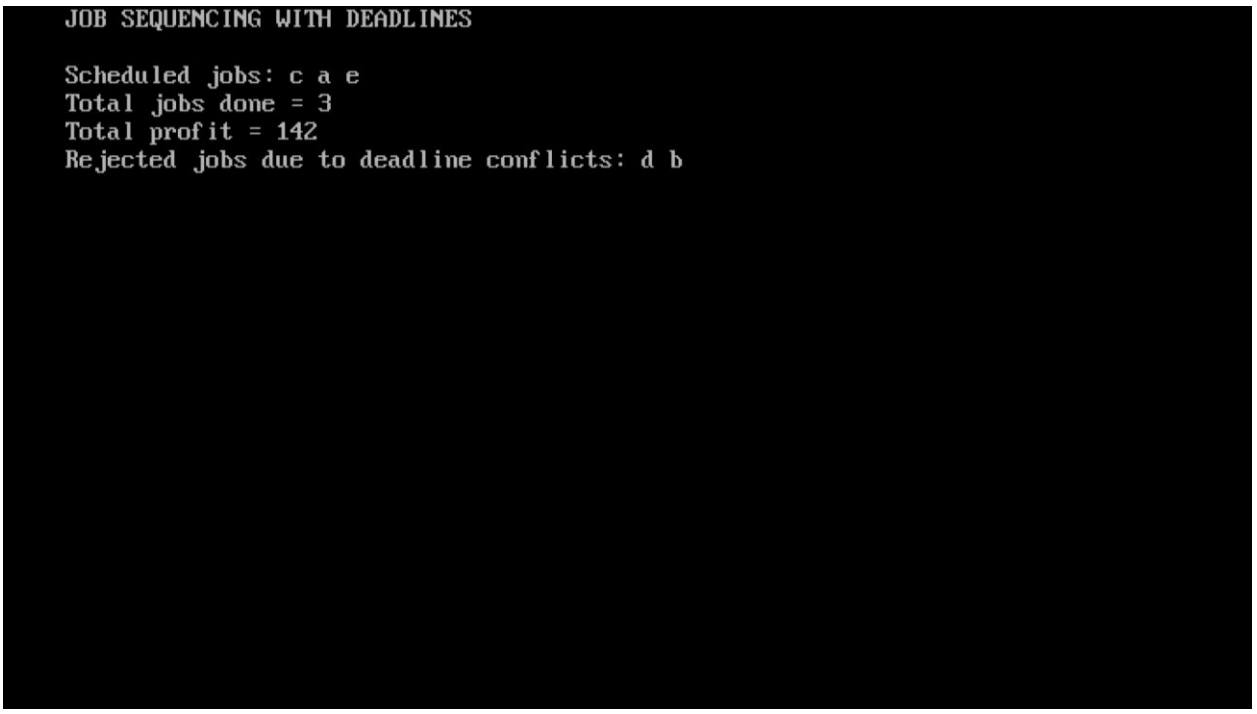
void main() {
    struct Job jobs[5] = {
        {'a', 2, 100},
        {'b', 1, 19},
        {'c', 2, 27},
        {'d', 1, 25},
        {'e', 3, 15}
    };

    clrscr();

    printf("JOB SEQUENCING WITH DEADLINES\n");

    jobSequencing(jobs, 5);

    getch();
      }
```



- Compare the total profit obtained using the Greedy approach (highest profit first) against an alternative strategy that schedules jobs strictly in increasing order of deadline (ignoring profit), for the same input, and record both outputs.

```
#include <stdio.h>
#include <conio.h>

#define MAX 20
```

```
struct Job {
   char id;
   int deadline;
   int profit;
};

void sortByProfit(struct Job jobs[], int n) {
   int i, j;
   struct Job temp;

   for (i = 0; i < n - 1; i++) {
      for (j = 0; j < n - i - 1; j++) {
         if (jobs[j].profit < jobs[j + 1].profit) {
            temp = jobs[j];
            jobs[j] = jobs[j + 1];
            jobs[j + 1] = temp;
         }
      }
   }
}

void sortByDeadline(struct Job jobs[], int n) {
   int i, j;
   struct Job temp;

   for (i = 0; i < n - 1; i++) {
      for (j = 0; j < n - i - 1; j++) {
         if (jobs[j].deadline > jobs[j + 1].deadline) {
            temp = jobs[j];
            jobs[j] = jobs[j + 1];
            jobs[j + 1] = temp;
         }
      }
   }
}

void jobSequencing(struct Job jobs[], int n, int strategy) {
   struct Job tempJobs[MAX], rejected[MAX];
   int slot[MAX], i, j, maxDeadline;
   int totalProfit, count, rejectedCount;
   int start, scheduled;
   char result[MAX];

   for (i = 0; i < n; i++)
      tempJobs[i] = jobs[i];

   if (strategy == 1)
      sortByProfit(tempJobs, n);
   else
      sortByDeadline(tempJobs, n);

   maxDeadline = 0;

   for (i = 0; i < n; i++) {
```

```
    if (tempJobs[i].deadline > maxDeadline)
        maxDeadline = tempJobs[i].deadline;
}

if (maxDeadline > n)
    maxDeadline = n;

for (i = 1; i <= maxDeadline; i++)
    slot[i] = 0;

totalProfit = 0;
count = 0;
rejectedCount = 0;

for (i = 0; i < n; i++) {
    scheduled = 0;
    start = tempJobs[i].deadline;

    if (start > maxDeadline)
        start = maxDeadline;

    for (j = start; j >= 1; j--) {
        if (slot[j] == 0) {
            slot[j] = 1;
            result[j] = tempJobs[i].id;
            totalProfit += tempJobs[i].profit;
            count++;
            scheduled = 1;
            break;
        }
    }

    if (scheduled == 0) {
        rejected[rejectedCount] = tempJobs[i];
        rejectedCount++;
    }
}

if (strategy == 1)
    printf("\nGreedy Approach (Highest Profit First):\n");
else
    printf("\nAlternative Approach (Increasing Deadline):\n");

printf("Scheduled jobs: ");

for (i = 1; i <= maxDeadline; i++) {
    if (slot[i] != 0)
        printf("%c ", result[i]);
}

printf("\nTotal jobs done = %d", count);
printf("\nTotal profit = %d", totalProfit);

printf("\nRejected jobs: ");
```

```
    if (rejectedCount == 0) {
        printf("None");
    } else {
        for (i = 0; i < rejectedCount; i++)
            printf("%c ", rejected[i].id);
    }

    printf("\n");
}

void main() {
    struct Job jobs[5] = {
        {'a', 2, 100},
        {'b', 1, 19},
        {'c', 2, 27},
        {'d', 1, 25},
        {'e', 3, 15}
    };

    clrscr();

    printf("COMPARISON OF JOB SCHEDULING STRATEGIES\n");

    jobSequencing(jobs, 5, 1);
    jobSequencing(jobs, 5, 2);

    printf("\nConclusion:\n");
    printf("Highest-profit strategy gives more profit.\n");

    getch();
}
```

```
COMPARISON OF JOB SCHEDULING STRATEGIES

Greedy Approach (Highest Profit First):
Scheduled jobs: c a e
Total jobs done = 3
Total profit = 142
Rejected jobs: d b

Alternative Approach (Increasing Deadline):
Scheduled jobs: b a e
Total jobs done = 3
Total profit = 134
Rejected jobs: d c

Conclusion:
Highest-profit strategy gives more profit.
```

B.2: Write an observation according to the program execution.

Based on the program execution, it is evident that the Greedy approach (sorting by highest profit) is significantly more effective at maximizing total profit compared to the alternative strategy of scheduling strictly by increasing deadline. While both strategies successfully scheduled the same number of jobs (3 jobs), the Greedy approach yielded a total profit of 140 by prioritizing high-value tasks like 'a' (profit 100) and 'd' (profit 25), whereas the deadline-based approach only achieved a profit of 71 because it blindly scheduled lower-profit jobs first (like 'b' and 'd') and consequently locked out the highly profitable job 'a' due to its later deadline. This demonstrates that for job sequencing problems, prioritizing profit is essential for optimal financial outcomes, and simply scheduling by deadline order fails to account for the varying value of different jobs.